Guía 1 de 3 — El proyecto de ejemplo, paso a paso

DPL1046 · Clase 8 · Para repasar en casa, a tu ritmo.

En la Clase 8 viste un proyecto integrador **completo**, de principio a fin. Esta guía te deja recorrerlo de nuevo con calma: qué problema resuelve, cómo está organizado, cómo correrlo y qué hace cada archivo. **No es para copiarlo**: es para entender cómo se ve un proyecto que cumple la rúbrica y comparar el tuyo.

> Antes de empezar, haz tu rutina de inicio (Guía 1 de la Clase 2) con el `.venv` activado y trae lo último con `git pull`.

---

1. El problema que resuelve

Un equipo de compras de un e-commerce necesita saber **cuántas ventas esperar en las próximas 8 semanas**. Sin ese pronóstico se compra de más (dinero congelado en inventario) o de menos (ventas perdidas).

El proyecto usa **dos fuentes de datos**:

- **Olist** (CSV): pedidos (99.441) e ítems de cada pedido (112.650).
- **API del dólar** (`mindicador.cl`): para pasar los precios de reales brasileños (BRL) a pesos chilenos (CLP).

> **Analogía:** es como una cocina. Primero traes los ingredientes (**extraer**), luego los lavas, cortas y cocinas (**transformar**) y al final sirves el plato (**cargar**). A esa secuencia se le llama **ETL**: *Extract*, *Transform*, *Load*.

2. Cómo está organizado

```
proyecto-ejemplo/
├── pipeline.py          ← el pipeline completo, en un solo archivo
├── test_pipeline.py     ← 10 pruebas con pytest
├── requirements.txt     ← librerías necesarias
└── README.md            ← cómo usarlo
```

Todo el pipeline vive en **un archivo**, ordenado en tres bloques (**extraer**, **transformar** y **cargar**). Cada función hace una sola cosa, y al final `main()` las llama en orden, como un jefe de cocina que dice "ahora tú, ahora tú". Arriba del archivo están los **parámetros** (la carpeta de datos, cuántas semanas pronosticar) en MAYÚSCULAS: si cambian, editas una línea.

3. Cómo correrlo

Con el entorno activo, **desde la raíz del repo** (como todos los scripts del curso):

```
pip install -r clases/clase-08/material/proyecto-ejemplo/requirements.txt
python clases/clase-08/material/proyecto-ejemplo/pipeline.py
pytest clases/clase-08/material/proyecto-ejemplo/ -v
```

Debes ver líneas `INFO` con lo que hace cada paso y, al final, un **RESUMEN PARA LA GERENCIA**. Después, `pytest` debe mostrar `10 passed`. Los resultados quedan en la carpeta `salida/`.

Si algo falla:

Problema	Qué hacer
`ERROR | No encontré los CSV de Olist`	Estás en otra carpeta o faltan los CSV. Corre el comando desde la **raíz del repo** y revisa que los CSV estén en `data/raw/` (los de la Clase 1).
`WARNING | La API del dólar no respondió`	**No es un error:** no hay internet o la API no respondió. El pipeline usa un valor de respaldo (1 BRL ≈ 180 CLP) y sigue.
`ModuleNotFoundError`	Falta instalar las librerías: `pip install -r clases/clase-08/material/proyecto-ejemplo/requirements.txt` con el `(.venv)` activo.

4. Recorrido por el código

Parámetros y `main()`: el jefe de cocina

```
RUTA_DATOS = "data/raw/"
SEMANAS_A_PRONOSTICAR = 8
...
pedidos, items = extraer_olist()
real_en_clp = extraer_dolar()
ventas_semanales = calcular_ventas_semanales(pedidos, items, real_en_clp)
ventas_mensuales = calcular_ventas_mensuales(ventas_semanales)
pronostico = pronosticar(ventas_semanales)
cargar({...}, CARPETA_SALIDA)
```

`main()` llama a cada paso en orden: extraer → transformar → cargar. Cada paso avisa qué hizo con `logging.info(...)`: es el **diario** del programa (Clase 7).

`extraer_dolar`: una API con plan B

```
        try:
            respuesta = requests.get("https://mindicador.cl/api/dolar", timeout=10)
            respuesta.raise_for_status()
            dolar_en_clp = respuesta.json()["serie"][0]["valor"]
            ...
        except requests.RequestException:
            logging.warning("La API del dólar no respondió: uso el valor de respaldo")
            return REAL_EN_CLP_RESPALDO
```

- `requests.get(url, timeout=10)` pide el dólar (como en la Clase 4); si tarda más de 10 segundos, se rinde.
- `raise_for_status()` avisa si la respuesta fue un error.
- `["serie"][0]["valor"]` entra al JSON: lista → diccionario → valor más reciente.
- `except requests.RequestException` atrapa **el error que esperamos** (la API no responde), lo **registra** y devuelve un **valor de respaldo**: el pipeline no se cae.

> **Analogía:** es el plan B del taxi. Si la app no funciona, llamo por teléfono.

`calcular_ventas_semanales`: limpiar, combinar y resumir

1. **Filtrar** solo los pedidos entregados: `pedidos[pedidos["order_status"] == "delivered"]` (Clase 5).
2. **Combinar** ítems con pedidos: `items.merge(entregados, on="order_id", how="inner")` (Clase 6).
3. **Columna nueva** con el precio en pesos: `tabla["ventas_clp"] = tabla["price"] * real_en_clp`.
4. **Fecha de la semana** (el lunes) con `pd.to_datetime(...)` y `.dt.to_period("W")` (Clase 5).
5. **Resumir** con `groupby("semana")["ventas_clp"].sum()` (Clase 6): una fila por semana.

`calcular_ventas_mensuales`: qué meses venden más

Otro `groupby`, ahora por mes del año: el **promedio semanal** de cada mes. El **índice** compara cada mes con el promedio general: `1,66` significa que ese mes vende un 66% más que una semana normal. Es una sola división.

`pronosticar`: un pronóstico simple

```
        promedio = ventas_semanales["ventas_clp"].tail(4).mean()
        ...
        for i in range(1, semanas + 1):
            fecha = ultima_semana + pd.Timedelta(weeks=i)
```

Toma el **promedio de las últimas 4 semanas** y lo repite hacia adelante, creando las semanas futuras con un ciclo `for` (Clase 7).

> El pronóstico es simple **a propósito**: un modelo sofisticado no es el objetivo del diplomado, sino un pipeline ordenado y confiable. En tu presentación, di con honestidad cuáles son sus límites.

`cargar`: guardar sin duplicar

```
conexion = sqlite3.connect(carpeta + "demanda.db")
for nombre, tabla in tablas.items():
    tabla.to_sql(nombre, conexion, if_exists="replace", index=False)
    tabla.to_csv(carpeta + nombre + ".csv", index=False)
```

- `sqlite3.connect(...)` abre (o crea) la base de datos (Clase 6).
- El ciclo guarda **cada tabla** en SQLite y deja una copia en CSV.
- `if_exists="replace"` **reescribe** la tabla: si corres el pipeline **dos veces**, el resultado es el mismo y **no se duplican filas**. Eso se llama **idempotencia** y está en la rúbrica.

`test_pipeline.py`: pruebas automáticas

Son 10 pruebas con `pytest`: **tres para cada función de transformación** (por ejemplo, que ignora los pedidos no entregados, que convierte a pesos y que suma por semana) y **una para la carga** (cargar dos veces no duplica filas). Cada una usa `assert` para **afirmar** algo que debe ser verdad, con datos pequeños donde ya sabemos la respuesta.

5. Qué punto de la rúbrica cubre cada pieza

Criterio (peso)	Dónde se ve en el ejemplo
Funcionalidad (35%)	Corre completo con un solo comando, integra CSV y API, resultados reproducibles.
Calidad del código (30%)	Funciones con docstring, `try/except`, logging, 10 tests.
Buenas prácticas (15%)	Parámetros ordenados al inicio, `requirements.txt`, README, carga sin duplicar, sin claves en el código.
Presentación (20%)	Problema claro, demo en vivo, valor de negocio. Mira la Guía 2.

6. Prueba tú mismo (10 minutos)

1. **Cambia un parámetro.** En `pipeline.py` cambia `SEMANAS_A_PRONOSTICAR = 8` por `12`, corre el pipeline y abre `salida/pronostico.csv`: ahora tiene 12 filas.
2. **Corre dos veces.** Las tablas quedan con las mismas filas: no se duplican.

3. **Sin internet.** Desconecta el wifi y corre de nuevo: verás el aviso de la API y el pipeline igual termina.
4. **Rompe un test a propósito.** En `test_pipeline.py` cambia el `200` esperado por `201`, corre `pytest clases/clase-08/material/proyecto-ejemplo/ -v` y lee con calma qué esperaba y qué obtuvo. Luego devuélvelo.

7. Preguntas de repaso

1. ¿Qué hace `main()` y por qué no calcula nada?
2. ¿Qué pasa si la API del dólar falla? ¿Dónde queda registrado?
3. ¿Por qué usa `replace` al guardar en SQLite?
4. Si quiero pronosticar 12 semanas en vez de 8, ¿qué línea cambio?
5. ¿Qué afirma `assert (pronostico["ventas_clp_estimado"] == 200).all()`?

Respuestas

1. Llama a las funciones en orden; el cálculo vive en cada función, así cada una tiene una sola responsabilidad.
2. Usa un valor de respaldo y lo registra con `logging.warning(...)`.
3. Porque reescribe la tabla completa: correr el pipeline dos veces da el mismo resultado, sin filas duplicadas.
4. `SEMANAS_A_PRONOSTICAR`, arriba de `pipeline.py`.
5. Que todas las semanas pronosticadas valen 200, el promedio de las últimas 4 semanas del ejemplo.